6. táblás gyakorlat feladatai

1. Gyűjtsük ki egy szekvenciális inputfájlban rendezve tárolt egész számok közül azt, hogy melyik számból hány darab található.

Specifikáció:

A = (x:infile($\mathbb{Z}$), y:outfile(Össz))
Össz = rec(szám:$\mathbb{Z}$, db:$\mathbb{N}$)

Ef = ($x = x_0 \wedge x\nearrow$)
(x↗ azt jelzi, hogy az x növekedően rendezett)

Új Specifikáció:

A = (t:enor(Össz), y:outfile(Össz))

Ef = ($t = t_0$)

Uf = ($y = t_0$) = ($y = \bigoplus_{e \in t_0} \langle e \rangle$)

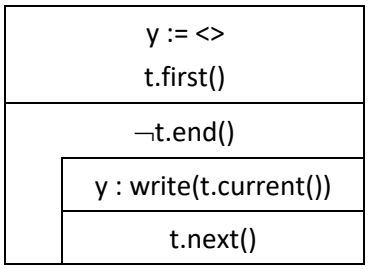
Összegzés (másolás)

f(e) ~ <e>
s ~ y
H, +, 0 ~ Össz*, $\oplus$, <>

Ötlet:

Ha lenne egy olyan felsorolónk, amelyik az eredményt, az összesítéseket tartalmazó rekordokat tudná felsorolni, akkor elég lenne ezeket az output fájlba átmásolni.

Algoritmus:



Felsoroló:

t:enor(Össz) Össz = rec(szám:$\mathbb{Z}$, db:$\mathbb{N}$)

Össz*	first()	next()	current() : Össz	end() :$\mathbb{L}$
x : infile($\mathbb{Z}$) dx : $\mathbb{Z}$ sx : Status akt : Össz vége : $\mathbb{L}$	sx,dx,x:read next()	lásd külön	**return** akt	**return** vége

next() művelet

A = (x:infile($\mathbb{Z}$), dx:$\mathbb{Z}$, sx:Status, akt:Össz, vége:$\mathbb{L}$)

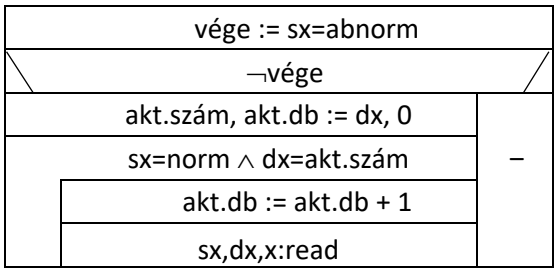
Ef = ($x = x' \wedge x\nearrow \wedge dx = dx' \wedge sx = sx'$)

Uf = (vége = (sx'=abnorm) $\wedge$ ($\neg$vége $\rightarrow$ akt.szám=dx' $\wedge$ (akt.db, (sx,dx,x))=$\sum_{dx \in (dx',x')}^{dx = akt.szám} 1$))

Megj: Az összegzésnek két eredménye van: a darabszám (akt.db); és a felsoroló aktuális állapota, amelyet az sx,dx,x változók értékei írnak le a next() művelet végén.

Összegzés (megszámolás)

t:enor(E) ~ x:infile($\mathbb{Z}$) (sx,dx,x:read)
first() nélkül, felt: dx=akt.szám
f(e) ~ 1
s ~ akt.db
H, +, 0 ~ $\mathbb{N}$, +, 0



2. Számoljuk meg egy karakterekből álló szekvenciális inputfájlban a szavakat úgy, hogy a *12* betűnél hosszabb szavakat duplán vesszük figyelembe! (Egy szót szóközök vagy a fájl vége határol.)

Specifikáció:

A = (x:infile($\mathbb{K}$), c:$\mathbb{N}$)
Ef = (x = x_0)

Új specifikáció:

A = (t:enor($\mathbb{N}$), c:$\mathbb{N}$)
Ef = (t = t_0)
Uf = (c = $\sum_{e \in t_0}$ {2, ha e>12; 1, ha e≤12})

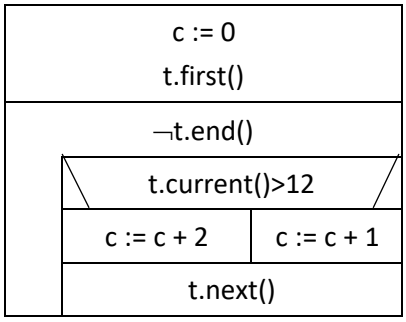
Összegzés

f(e) ~ {2, ha e>12; 1, ha e≤12}
s ~ c
H, +, 0 ~ $\mathbb{N}$, +, 0

Ötlet: 1.kvíz

Soroljuk fel csak a szavak hosszait, és ebből az eredményt már könnyen kiszámolhatjuk.

Algoritmus:



Felsoroló:

t:enor($\mathbb{N}$)

$\mathbb{N}$*	first()	next()	current() : $\mathbb{N}$	end() : $\mathbb{L}$
x : infile($\mathbb{K}$) dx : $\mathbb{K}$ sx : Status akt : $\mathbb{N}$ vége : $\mathbb{L}$ 2.kvíz	sx,dx,x:read next()	lásd külön	**return** akt	**return** vége

next() művelet

A = (x:infile($\mathbb{K}$), dx:$\mathbb{K}$, sx:Status, akt:$\mathbb{N}$, vége:$\mathbb{L}$)
Ef = (x = x' ∧ dx = dx' ∧ sx = sx') 3.kvíz [Uf]
Uf = ((dx",(sx",dx",x"))=SELECT$_{dx \in (dx',x')}$ (sx=abnorm ∨ dx≠' ') ∧ $_{dx \neq ' '}$
vége=(sx"=abnorm) ∧ (¬vége → (akt, (sx,dx,x))=$\sum_{dx \in (dx",x")}$ 1))

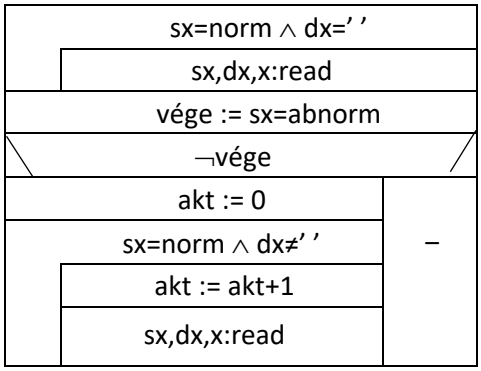
Megj: A kiválasztásnak két eredménye van: a keresett szóköz (dx"), amelyhez nem vezetünk be külön output változót; és a felsoroló aktuális állapota, amelyet az (sx",dx",x") hármas ír le. (Nem okozna félreértést, ha csak az (sx",dx",x")-t tüntetnénk fel a kiválasztás eredményeként.)

Kiválasztás

t:enor(E) ~ x:infile($\mathbb{K}$) (sx,dx,x:read)
first() nélkül
felt(e) ~ sx=abnorm ∨ dx≠' '

Összegzés (megszámolás)

t:enor(E) ~ x:infile($\mathbb{K}$) (sx,dx,x:read)
first() nélkül, felt: dx≠' '
f(e) ~ 1
s ~ akt
H, +, 0 ~ $\mathbb{N}$, +, 0



4.kvíz [alternatív megoldás]

3. Egy karakterekből álló szekvenciális inputfájlban melyik az a leghosszabb szó, amelyik tartalmazza a 'w' betűt? (Egy szót szóközök vagy a fájl vége határol.)

Specifikáció:

A = (x:infile($\mathbb{K}$), l:$\mathbb{L}$, szó:$\mathbb{S}$)

Ef = (x = x_0)

Ötlet:

Soroljuk fel a szavakat, azzal az információval együtt, hogy van-e bennük 'w' betű.

Új Specifikáció:

A = (t:enor(Szó), l:$\mathbb{L}$, szó:$\mathbb{S}$)

Szó = rec(szó:$\mathbb{S}$, w:$\mathbb{L}$)

Ef = (t = t_0)

Uf = (((l, max, elem) = $\mathbf{MAX}_{e \in t_0}^{e.w}$ |e.szó|) $\wedge$ (l $\rightarrow$ szó=elem.szó))

Feltételes maximum keresés

f(e)	~	|e. szó|
felt(e)	~	e.w
H, >	~	$\mathbb{N}$, >

Algoritmus:

<table>
<tr><td colspan="4">l := hamis; t.first()</td></tr>
<tr><td colspan="4">¬t.end()</td></tr>
<tr><td></td><td>¬ t.current().w</td><td>t.current().w ∧ l</td><td>t.current().w ∧ ¬l</td></tr>
<tr><td></td><td rowspan="2">—</td><td>|t.current().szó| > max</td><td>l, max, szó := igaz,
|t.current().szó|, t.current().szó</td></tr>
<tr><td></td><td>max, szó :=
|t.current().szó|, t.current().szó</td><td>—</td></tr>
<tr><td colspan="4">t.next()</td></tr>
</table>

Felsoroló:

t:enor(Szó) Szó = rec(szó:$\mathbb{S}$, w:$\mathbb{L}$)

Szó*	first()	next()	current() : Szó	end() : $\mathbb{L}$
x : infile($\mathbb{K}$) dx : $\mathbb{K}$ sx : Status akt : Szó vége : $\mathbb{L}$	sx,dx,x:read next()	lásd külön	**return** akt	**return** vége

next() művelet

A = (x:infile($\mathbb{K}$), dx:$\mathbb{K}$, sx:Status, akt:Szó, vége:$\mathbb{L}$)

Ef = (x = x' $\wedge$ dx = dx' $\wedge$ sx = sx')

Uf = ((dx",(sx",dx",x"))=SELECT$_{dx \in (dx',x')}$ (sx=abnorm $\vee$ dx≠' ')

$\wedge$ vége=(sx"=abnorm)

$\wedge$ (¬vége $\rightarrow$ (akt.szó, (sx,dx,x)) = $\bigoplus_{dx \in (dx",x")}^{dx \neq ' '}$<dx> $\wedge$ (akt.w, (sx,dx,x)) =$\bigvee_{dx \in (dx",x")}^{dx \neq ' '}$dx='w'))

Megj: A „van 'w' karakter egy szóban" részfeladatot lineáris keresés helyett összevagyolással számoljuk ki, mert így is, úgy is végig kell menni a teljes aktuális szón (lásd a szó betűinek összefűzését), amellyel az összevagyolást össze lehet vonni közös felsorolásba (ciklusba).

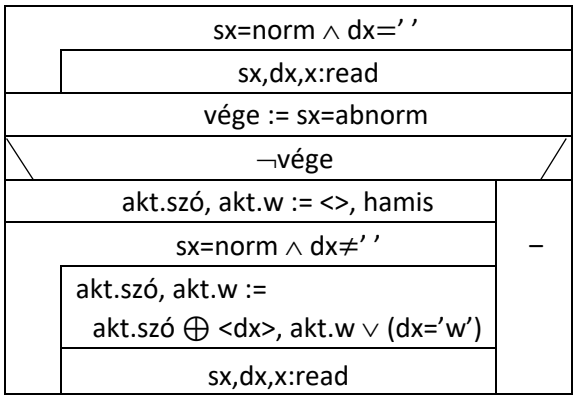
5.kvíz [milyen algoritmus mintára, és hogyan vezessük vissza a next() művelet másidik felét?]

Kiválasztás

t:enor(E) ~ x:infile($\mathbb{K}$) (sx,dx,x:read)
first() nélkül
felt(e) ~ sx=abnorm $\vee$ dx$\neq$' '

Két összegzés (összefűzés és összevagyolás)

t:enor(E) ~ x:infile($\mathbb{K}$) (sx,dx,x:read)
first() nélkül, felt: dx$\neq$' '
f(e) ~ (<dx>, dx='w')
s ~ (akt.szó, akt.w)
H, +, 0 ~ ($\mathbb{K}$*, $\mathbb{L}$), ($\bigoplus$, $\vee$), (<>, hamis)



4. Egy szekvenciális inputfájlban egy banknál számlát nyitott ügyfelek e havi kivét/betét forgalmát (tranzakcióit) tároljuk. Egy tranzakció tartalmazza az ügyfél számlaszámát, a tranzakció dátumát, és az összegét (előjeles egész szám: negatív a kivét, pozitív a betét). A tranzakciók a szekvenciális fájlban számlaszám szerint rendezetten helyezkednek el. Gyűjtsük ki azon számlaszámokat, és az ahhoz tartozó tranzakcióknak az egyenlegét, ahol ez az egyenleg kisebb –100000 Ft-nál.

Specifikáció:

A = (x:infile(Tranz), y:outfile(Egyenleg))

Tranz = rec(száml:$\mathbb{S}$, dátum:$\mathbb{S}$, össz:$\mathbb{Z}$)

Egyenleg = rec(száml:$\mathbb{S}$, egy:$\mathbb{Z}$)

Ef = ($x = x_0 \wedge x\nearrow_{száml}$)

(az $x\nearrow_{száml}$ azt jelzi, hogy x számlaszám szerint növekedően rendezett)

Új specifikáció:

A = (t:enor(Egyenleg), y:outfile(Egyenleg))

Ef = ($t = t_0$)

Uf = ($y = \bigoplus_{e \in t_0, e.egy<-100000} <e>$)

Összegzés (kiválogatás)

f(e) ~ <e> ha e.egy < -100000

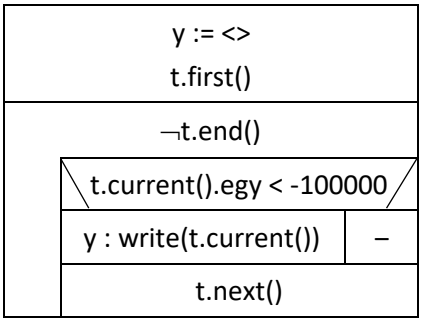
s ~ y

H, +, 0 ~ Egyenleg*, $\oplus$, <>

Ötlet:

Soroljuk fel számlaszámonként a transzakciók összesített egyenlegeit, hogy ezekből kiválogathassuk a -100000-nél kisebb egyenlegűeket.

Algoritmus:



Felsoroló:

t:enor(Egyenleg) Egyenleg = rec(száml:$\mathbb{S}$, egy:$\mathbb{Z}$)

Egyenleg*	first()	next()	current() : Egyenleg	end() : $\mathbb{L}$
x : infile(Tranz) dx : Tranz sx : Status akt : Egyenleg vége : $\mathbb{L}$	sx,dx,x:read next()	lásd külön	**return** akt	**return** vége

Tranz = rec(száml:$\mathbb{S}$, dátum:$\mathbb{S}$, össz:$\mathbb{Z}$)

next() művelet

A = (x:infile(Tranz), dx:Tranz, sx:Status, akt:Egyenleg, vége:$\mathbb{L}$)

Ef = ($x = x' \wedge x\nearrow_{száml} \wedge dx = dx' \wedge sx = sx'$)

Uf = (vége=(sx'=abnorm)$\wedge$(¬vége $\rightarrow$ akt.száml=dx'.száml $\wedge$ (akt.egy, (sx,dx,x))=$\sum_{dx \in (dx',x'),\ dx.száml = akt.száml}$ dx.össz))

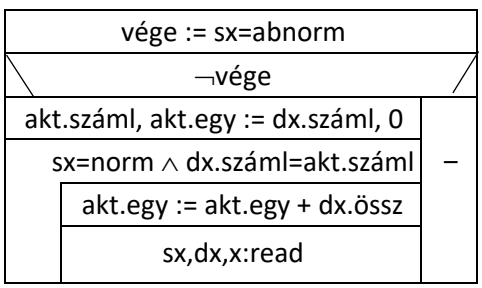
Összegzés

t:enor(E) ~ x:infile(Ügyfél)) (sx,dx,x:read)
first() nélkül,
felt: dx.száml=akt.száml

f(e) ~ dx.össz

s ~ akt.egy

H, +, 0 ~ $\mathbb{Z}$, + , 0



5. Egy szekvenciális inputfájlban a vadászok eredményeit (vadász neve, vadászat dátuma, zsákmány fajtája, zsákmány súlya) tároljuk a vadászok neve szerint rendezetten. Igaz-e, hogy minden vadász valamelyik vadászatán lőtt medvét?

Specifikáció:

A = (x:infile(Trófea), l:$\mathbb{L}$)

Trófea =

rec(név:$\mathbb{S}$, dátum:$\mathbb{S}$, fajta:$\mathbb{S}$, súly:$\mathbb{N}$)

Ef = ($x=x_0 \wedge x\nearrow_{név}$)

Optimista lineáris keresés

t:enor(E) ~ t:enor(Vadász)

felt(e) ~ e

Új specifikáció:

A = (t:enor($\mathbb{L}$), l:$\mathbb{L}$)

Ef = ($t=t_0$)

Uf = ($l = \forall SEARCH_{e \in t_0} e$)

Ötlet:

Soroljunk fel annyi logikai értéket ahány vadász van: egy logikai érték akkor igaz, ha a vadász lőtt medvét.

Algoritmus:

l := igaz t.first()	
$l \wedge \neg$t.end()	
	l := t.current()
	t.next()

Felsoroló:

t:enor($\mathbb{L}$)

$\mathbb{L}$*	first()	next()	current() : $\mathbb{L}$	end() : $\mathbb{L}$
x : infile(Trófea) dx : Trófea sx : Status akt : $\mathbb{L}$ vége : $\mathbb{L}$	sx,dx,x:read next()	lásd külön	**return** akt	**return** vége

Trófea = rec(név:$\mathbb{S}$, dátum:$\mathbb{S}$, fajta:$\mathbb{S}$, súly:$\mathbb{N}$)

next() művelet

A = (x:infile(Trófea), dx:Trófea, sx:Status, akt:$\mathbb{L}$, vége:$\mathbb{L}$)

Ef = ($x = x' \wedge x\nearrow_{név} \wedge dx = dx' \wedge sx = sx'$)

Uf = (vége = (sx'=abnorm) $\wedge$ ($\neg$vége $\rightarrow$ (akt, (sx,dx,x)) $= \bigvee_{dx \in (dx', x')}^{dx.név=dx'.név}$ dx.fajta="medve"))

Összegzés (összevagyolás)

t:enor(E) ~ x:infile(Trófea) (sx,dx,x:read)
first() nélkül,
felt: dx.név=dx'.név

f(e) ~ dx.fajta="medve"

s ~ akt

H, +, 0 ~ $\mathbb{L}$, $\vee$, hamis

vége := sx=abnorm		
$\neg$vége		
akt := hamis		–
név := dx.név		
sx=norm $\wedge$ dx.név=név		
	akt := akt $\vee$ (dx.fajta="medve")	
	sx,dx,x:read	

6. Egy szekvenciális inputfájlban a vadászok eredményeit (vadász neve, vadászat dátuma, zsákmány fajtája, zsákmány súlya) tároljuk a vadászok neve, azon belül a vadászat dátuma szerint rendezetten. Hány olyan vadász volt, aki minden vadászatán lőtt nyulat, és a zsákmányainak összsúlya meghaladta a 250 kilogrammot.

Specifikáció:

A = (x:infile(Trófea), c: $\mathbb{N}$)

Trófea =

rec(név:$\mathbb{S}$, dátum:$\mathbb{S}$, fajta:$\mathbb{S}$, súly:$\mathbb{N}$)

Ef = ($x=x_0 \wedge x\nearrow_{(\text{név,dátum})}$)

Új specifikáció:

A = (t:enor(Vadász), c: $\mathbb{N}$)

Vadász = rec(név: $\mathbb{S}$, nyúl:$\mathbb{L}$, összsúly:$\mathbb{N}$)

Ef = ($t=t_0$)

Uf = ($c = \sum_{e \in t_0} 1$)

e.nyúl $\wedge$ e.összsúly>250

Számlálás

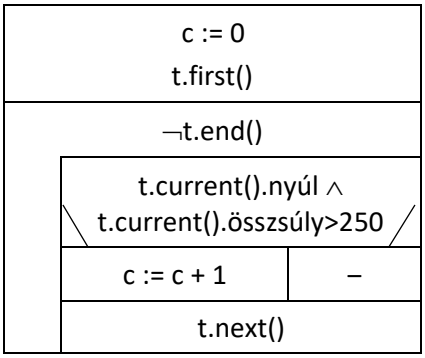
t:enor(E) ~ t:enor(Vadász)

felt(e) ~ e.nyúl $\wedge$ e.összsúly>250

Ötlet:

Soroljuk fel a vadászok neveit azzal az információval együtt, hogy lőttek-e minden vadászatukon nyulat, és mennyi volt a zsákmányaiknak az összsúlya.

Algoritmus:



Felsoroló:

Ötlet: A t:enor(Vadász) felsoroló megvalósításához tegyük fel, hogy rendelkezünk olyan y:enor(Összesítés) felsorolóval, amely a vadászoknak az egyes vadászatokon elért összesített eredményeit sorolja fel a vadász neve és a vadászat dátuma szerint növekedően rendezetten: lőtt-e egy adott vadászatán nyulat, és mennyi volt ott a zsákmányai súlyának összege.

t:enor(Vadász) Vadász = rec(név: $\mathbb{S}$, nyúl:$\mathbb{L}$, összsúly:$\mathbb{N}$)

Vadász*	first()	next()	current() : Vadász	end() : $\mathbb{L}$
y : enor(Összesítés) akt : Vadász vége : $\mathbb{L}$	y.first() next()	lásd külön	**return** akt	**return** vége

Összesítés = rec(név: $\mathbb{S}$, dátum:$\mathbb{S}$, lőtt:$\mathbb{L}$, össz:$\mathbb{N}$)

next() művelet

A = (y:enor(Összesítés), akt:Vadász, vége:$\mathbb{L}$)

Ef = (y = y' $\wedge$ (y folyamatban van $\vee$ y befejeződött) $\wedge$ $y\nearrow_{(\text{név,dátum})}$)

Uf = (vége = y'.end() $\wedge$ (¬vége $\rightarrow$ akt.név = y'.current().név

$\wedge$ (akt.nyúl, y) $= \bigwedge_{e \in (y'.\text{current}(), y')}^{e.\text{név}=\text{akt.név}}$ e.lőtt

$\wedge$ (akt.összsúly, y) $= \sum_{e \in (y'.\text{current}(), y')}^{e.\text{név}=\text{akt.név}}$ e. össz))

Két összegzés (éselés és összeadás)

közös felsorolóval

t:enor(E) ~ y:enor(Összesítés)
first() nélkül
felt: y.current().név=akt.név

f(e) ~ (y.current().lőtt, y.current().össz)

s ~ (akt.nyúl, akt.összsúly)

H, +, 0 ~ ($\mathbb{L}$, $\mathbb{N}$), ($\wedge$, +), (igaz, 0)

vége := y.end()	
¬vége	
akt := (y.current().név, igaz, 0)	–
¬y.end() $\wedge$ y.current().név=akt.név	
akt.nyúl := akt.nyúl $\wedge$ y.current().lőtt	
akt.összsúly := akt.összsúly + y.current().össz	
y.next()	

Felsoroló:

y:enor(Összesítés) Összesítés = rec(név: $\mathbb{S}$, dátum:$\mathbb{S}$, lőtt:$\mathbb{L}$, össz:$\mathbb{N}$)

Összesítés*	first()	next()	current() : Összesítés	end() : $\mathbb{L}$
x : infile(Trófea) dx : Trófea sx : Status akt : Összesítés vége : $\mathbb{L}$	sx,dx,x:read next()	lásd külön	**return** akt	**return** vége

Trófea = rec(név:$\mathbb{S}$, dátum:$\mathbb{S}$, fajta:$\mathbb{S}$, súly:$\mathbb{N}$)

next() művelet

A = (x:infile(Trófea), dx:Trófea, sx:Status, akt:Összesítés, vége:$\mathbb{L}$)

Ef = (x = x' $\wedge$ $x\nearrow_{(név,dátum)}$ $\wedge$ dx = dx' $\wedge$ sx = sx')

Uf = (vége = (sx'=abnorm) $\wedge$ (¬vége $\rightarrow$ akt.név=dx'.név $\wedge$ akt.dátum=dx'.dátum $\wedge$

$$\wedge\ (akt.lőtt, (sx,dx,x)) = \bigvee_{dx \in (dx',x')}^{dx,név=akt.név \wedge dx,dátum=akt.dátum} dx.fajta=\text{"nyúl"} \wedge$$

$$\wedge\ (akt.össz, (sx,dx,x)) = \sum_{dx \in (dx',x')}^{dx,név=akt.név \wedge dx,dátum=akt.dátum} dx.súly \quad) \quad)$$

Két összegzés (vagyolás és összeadás)

közös felsorolóval

t:enor(E) ~ x:infile(Trófea)
sx,dx,x:read, first() nélkül,
felt: dx.név=akt.név
$\wedge$ dx.dátum=akt.dátum

f(e) ~ (dx.fajta="nyúl", dx.súly)

s ~ (akt.lőtt, akt.össz)

H, +, 0 ~ ($\mathbb{L}$, $\mathbb{N}$), ($\vee$, +), (hamis, 0)

vége := sx=abnorm	
¬vége	
akt := (dx.név, dx.dátum, hamis, 0)	–
sx=norm $\wedge$ dx.név=akt.név $\wedge$ dx.dátum=akt.dátum	
akt.lőtt := akt.lőtt $\vee$ (dx.fajta="nyúl")	
akt.össz := akt.össz + dx.súly	
sx,dx,x:read	

7. Másoljuk át karakterenként egy szekvenciális inputfájl szövegét egy szekvenciális outputfájlba úgy, hogy a szavak között csak egyetlen szóközt tartunk meg.

Specifikáció:

A = (x:infile($\mathbb{K}$), y:infile($\mathbb{K}$))

Ef = ($x = x_0$)

Ötlet:

Soroljuk fel a szavakat, és írjuk ki úgy őket, hogy egyetlen szóközt teszünk csak közéjük.

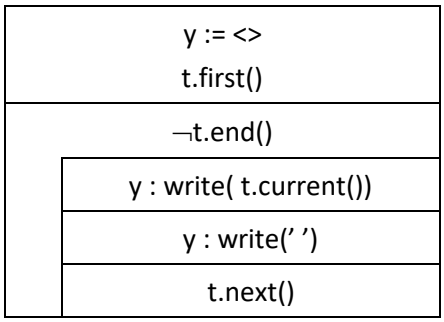
Új specifikáció:

A = (t:enor($\mathbb{S}$), y:infile($\mathbb{K}$))

Ef = ($t = t_0$)

Uf = ($y = \bigoplus_{e \in t_0} (e \oplus <' '>)$)

Algoritmus:



Összegzés (összefűzés)

f(e) ~ e ⊕ <' '>
s ~ y
H, +, 0 ~ $\mathbb{K}^*$, ⊕, <>

Felsoroló

t:enor($\mathbb{S}$)

$\mathbb{S}^*$	first()	next()	current() : $\mathbb{S}$	end() : $\mathbb{L}$
x : infile($\mathbb{K}$) dx : $\mathbb{K}$ sx : Status akt : $\mathbb{S}$ vége : $\mathbb{L}$	sx,dx,x:read szóközt_átlép() next()	lásd külön	**return** akt	**return** vége

szóközt_átlép() művelet

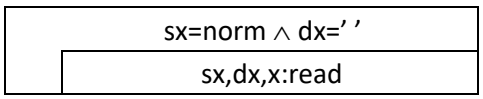
A = (x:infile($\mathbb{K}$), dx:$\mathbb{K}$, sx:Status)

Ef = ($x = x' \wedge dx = dx' \wedge sx = sx'$)

Uf = ((sx,dx,x) = SELECT$_{dx \in (dx',x')}$(sx=abnorm$\vee$dx≠' '))

Kiválasztás

t:enor(E) ~ x:infile($\mathbb{K}$) (sx, dx, x:read)
first nélkül
felt(e) ~ sx=abnorm $\vee$ dx≠' '



next() művelet

A = (x:infile($\mathbb{K}$), dx:$\mathbb{K}$, sx:Status, akt:$\mathbb{S}$, vége:$\mathbb{L}$)

Ef = ($x = x' \wedge dx = dx' \wedge sx = sx' \wedge$ (sx=norm $\rightarrow$ dx≠' ')) $_{dx \neq ' '}$

Uf = (vége = (sx'=abnorm) $\wedge$ (¬vége $\rightarrow$ (akt, (sx",dx",x"))=$\bigoplus_{dx \in (dx',x')}$ <dx>
$\wedge$ (sx,dx,x)= SELECT$_{dx \in (dx',x')}$(sx=abnorm$\vee$dx≠' ')))

Összegzés (összefűzés)

t:enor(E) ~ x:infile($\mathbb{K}$) (sx,dx,x:read)
first() nélkül, felt: dx≠' '
f(e) ~ <dx>
s ~ akt
H, +, 0 ~ $\mathbb{K}^*$, ⊕, <>

